STORAGE STRUCTURES, TRANSACTION MANAGEMENT, AND CONCURRENCY CONTROL

File Organisation, Organisation of Records in Files. Indexing and Hashing; Transaction Concepts — State, Serializability, Recoverability; Concurrency Control — Locks, Two-Phase Locking, Deadlock Handling. Transaction Management in Multi-Databases.

File Organization

File organization in DBMS refers to the method of storing data records in a file so they can be accessed efficiently. It determines how data is arranged, stored, and retrieved from physical storage.

The Objective of File Organization

- It helps in the faster selection of records i.e. it makes the process faster.
- Different Operations like inserting, deleting, and updating different records are faster and easier.
- It prevents us from inserting duplicate records via various operations.
- It helps in storing the records or the data very efficiently at a minimal cost.

Types of File Organizations

Various methods have been introduced to Organize files. These particular methods have advantages and disadvantages on the basis of access or selection. Thus it is all upon the programmer to decide the best-suited file Organization method according to his requirements.

Some types of File Organizations are:

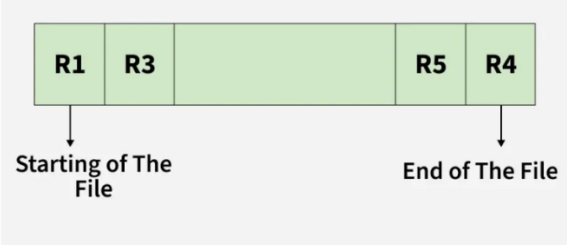
- Sequential File Organization
- Heap File Organization
- Clustered File Organization
- ISAM (Indexed Sequential Access Method)
- Hash File Organization
- B+ Tree File Organization

Sequential File Organization

The easiest method for file Organization is the Sequential method. In this method, the file is stored one after another in a sequential manner. There are two ways to implement this method:

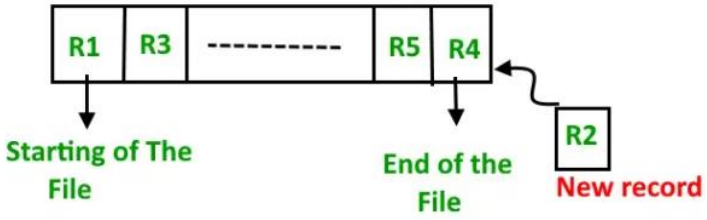
1. Pile File Method

This method is quite simple, in which we store the records in a sequence i.e. one after the other in the order in which they are inserted into the tables.



Pile File Method

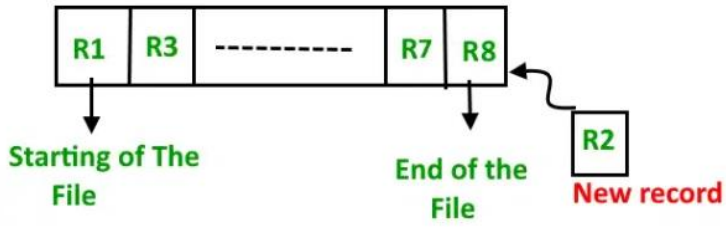
Insertion of the new record: Let the R1, R3, and so on up to R5 and R4 be four records in the sequence. Here, records are nothing but a row in any table. Suppose a new record R2 has to be inserted in the sequence, then it is simply placed at the end of the file.



New Record Insertion

Sorted File Method

In this method, As the name itself suggests whenever a new record has to be inserted, it is always inserted in a sorted (ascending or descending) manner. The sorting of records may be based on any primary key or any other key.



Sorted File Method

Insertion of the new record: Let us assume that there is a preexisting sorted sequence of four records R1, R3, and so on up to R7 and R8. Suppose a new record R2 has to be inserted in the sequence, then it will be inserted at the end of the file and then it will sort the sequence.



new Record Insertion

Advantages of Sequential File Organization

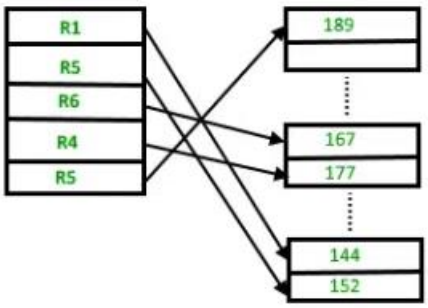
- Fast and efficient method for huge amounts of data.
- Simple design.
- Files can be easily stored inmagnetic tapes i.e. cheaper storage mechanism.

Disadvantages of Sequential File Organization

- Time wastage as we cannot jump on a particular record that is required, but we have to move in a sequential manner which takes our time.
- The sorted file method is inefficient as it takes time and space for sorting records.

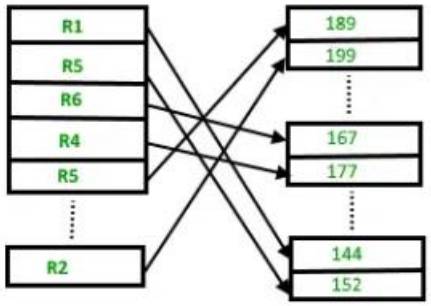
Heap File Organization

Heap File Organization works with data blocks. In this method, records are inserted at the end of the file, into the data blocks. No Sorting or Ordering is required in this method. If a data block is full, the new record is stored in some other block, Here the other data block need not be the very next data block, but it can be any block in the memory. It is the responsibility of DBMS to store and manage the new records.



Heap File Organization

Insertion of the new record: Suppose we have four records in the heap R1, R5, R6, R4, and R3, and suppose a new record R2 has to be inserted in the heap then, since the last data block i.e data block 3 is full it will be inserted in any of the data blocks selected by the DBMS, let's say data block 1.



New Record Insertion

Advantages of Heap File Organization

- Fetching and retrieving records is faster than sequential records but only in the case of small databases.
- When there is a huge number of data that needs to be loaded into the database at a time, then this method of file Organization is best suited.

Disadvantages of Heap File Organization

- The problem of unused memory blocks.
- Inefficient for larger databases.

Key Features of Heap File Organization

Heap file organization (also called unordered file organization) is the simplest way of storing records in a database file.

- Unordered storage: Records are placed in no particular order. A new record goes into whichever block has free space, usually the end of the file.
- Fast insertion: Insertion is very efficient. The DBMS just locates the last block (or any block with free space) and appends the record. No sorting or searching is needed.
- No ordering or key requirement: There is no sort key or hash function, so the physical order of records has no relation to attribute values.
- Slow searching: To find a record, the DBMS must perform a linear scan through the blocks. The cost is O(n) in the worst case, and on average about half the file is read for a successful search.
- Inefficient range and ordered queries: Because records are unsorted, range queries (e.g., salary BETWEEN 20000 AND 40000) and ORDER BY operations require scanning the entire file, and sorting if needed.
- Deletion leaves free space: Deleting a record leaves a gap in its block. This space can be reused by later insertions, but it can lead to internal fragmentation unless the file is periodically reorganized.
- Simple structure with low overhead: There is no index maintenance or ordering logic, so the file is easy to implement and manage.
- Efficient for full-table scans and bulk loading: When all records must be processed (e.g., reports, batch operations) or when large volumes of data are loaded quickly, a heap file performs well.
- Works well with secondary indexes: Heap files are often paired with indexes (such as B+ trees) to speed up searches while keeping insertion cheap.
- Updates: If an updated record is the same size, it is modified in place. If it grows larger, it may need to move to another block.

Heap File Organization: Case Study

Scenario: Hospital Patient Registration System

A city hospital registers hundreds of patients every day at its reception desk. Each registration creates a record in a PATIENT table. Patients arrive in no particular order, and the primary need is to record them quickly. The data is later used for billing reports and audits.

Table structure:

PATIENT(PatientID, Name, Age, Phone, Department, AdmitDate)

Assumptions:

Each block holds 3 records (kept small for demonstration).

The DBMS stores the table as a heap file.

Step 1: Insertion (Records Arrive in Any Order)

Patients register in this order:

Arrival Order	PatientID	Name	Department
1	205	Anita	Cardiology
2	101	Ravi	Orthopedics
3	330	Meera	Pediatrics
4	150	Joseph	Cardiology
5	412	Sara	ENT
6	120	Kiran	Neurology
7	275	Binu	Orthopedics

Resulting file layout on disk:

Block 1: [205 Anita] [101 Ravi] [330 Meera]

Block 2: [150 Joseph] [412 Sara] [120 Kiran]

Block 3: [275 Binu] [free] [free]

Observation: Records sit exactly in arrival order. There is no sorting by PatientID or name. Each new record simply went to the last block with free space. Insertion cost is about 1 block read + 1 block write, regardless of table size.

Step 2: Searching

Query: SELECT * FROM PATIENT WHERE PatientID = 120;

Since the file is unordered, the DBMS scans from Block 1:

Read Block 1: 205, 101, 330 (not found)

Read Block 2: 150, 412, 120 (found)

Cost: 2 block accesses here. For a hospital with 1,000,000 patients stored in 100,000 blocks, the worst case is 100,000 block reads, and the average is about 50,000.

Step 3: Range Query

Query: SELECT * FROM PATIENT WHERE PatientID BETWEEN 100 AND 200;

Matching records (101, 150, 120) are scattered across Blocks 1 and 2. The DBMS must scan all 3 blocks, because it cannot know where matches end. There is no early stopping.

Step 4: Deletion

Operation: Patient 412 (Sara) is discharged and the record is removed.

Block 2: [150 Joseph] [free] [120 Kiran]

The slot becomes free. When Patient 310 registers next, the DBMS may place the record in this empty slot, reusing the space:

Block 2: [150 Joseph] [310 Nikhil] [120 Kiran]

If no new patients arrive, the gap stays as wasted space (internal fragmentation) until the file is reorganized.

Step 5: Full Table Scan (Where Heap Excels)

Query: SELECT Department, COUNT(*) FROM PATIENT GROUP BY Department;

This report needs every record anyway, so reading all blocks sequentially is the optimal approach. No ordering is required, so the heap file performs as well as any other organization.

Cost Analysis (Table of b = 100,000 Blocks)

Operation Heap File Cost

Insert ~2 block accesses

Search by PatientID ~b/2 = 50,000 (avg)

Range query b = 100,000

Full scan report b = 100,000

Delete (after locating) search cost + 1 write

Improving the Design

The hospital front desk needs fast lookups when a patient returns. The solution is to keep the heap file and add a B+ tree index on PatientID:

Insertions remain cheap (append to heap, then update the index).

Search drops from ~50,000 accesses to about 3 to 4 (the index height).

Indexing in Databases

Indexing in databases is a data structure technique used to speed up data retrieval operations by minimizing the number of disk accesses required to locate records.

- Stores copies of selected column (search key) values
- Maintains pointers to the actual data rows in the main table
- Allows faster searching without scanning the entire table



Attributes of Indexing

Several important attributes of indexing affect the performance and efficiency of database operations:

1. **Access Types:** This refers to the type of access, such as value-based search, range access, etc.
2. **Access Time:** Refers to the time needed to find a particular data element or set of elements.
3. **Insertion Time:** To the time taken to find the appropriate space and insert new data.
4. **Deletion Time:** Time taken to find an item and delete it as well as update the index structure.
5. **Space Overhead:** It refers to the additional space required by the index.

Features

- **Efficient Data Structures**: Indexes use efficient data structures like B-trees, B+ trees, and hash tables to enable fast data retrieval.
- **Periodic Index Maintenance**: Maintenance tasks include updating, rebuilding, or removing obsolete indexes.
- **Query Optimization**: The DBMS query optimizer uses indexes to determine the most efficient execution plan for a query.
- **Handling Fragmentation**: Regular defragmentation can help maintain optimal performance.

File Organization in Indexing

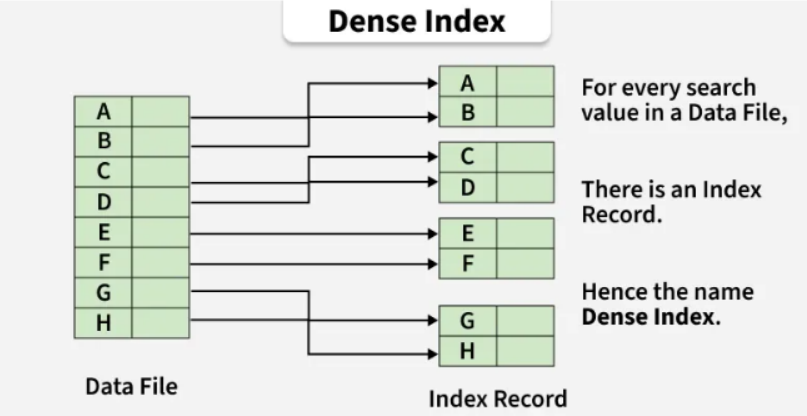
File organization refers to how data and indexes are physically stored in memory or on disk. The following are the common types of file organizations used in indexing:

1. Sequential (Ordered) File Organization

In this type of organization, the indices are based on a sorted ordering of the values. These are generally fast and a more traditional type of storing mechanism. These Ordered or Sequential file organizations might store the data in a dense or sparse format.

i. Dense Index: A dense index does not imply multiple identical key entries. Instead, a dense index maintains one index entry for every search key value present in the data file.

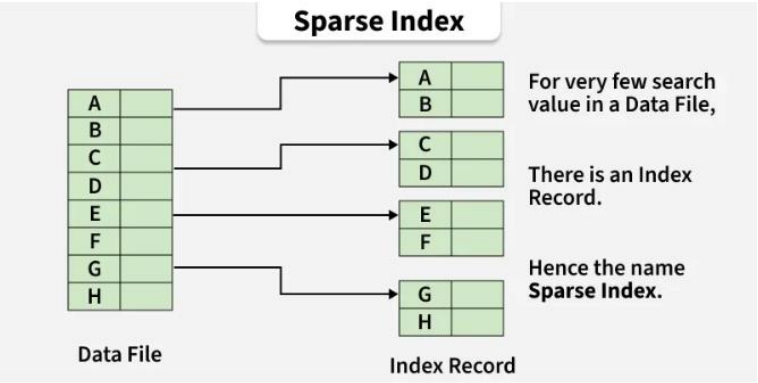
Example: If a table contains duplicate key values, a dense index maintains an index entry for every search key value present in the data file.



ii. Sparse Index: The index record appears only for a few items in the data file. Each item points to a block as shown. To locate a record, we find the index record with the largest search key value less than or equal to the search key value we are looking for.

Access Method: To locate a record, we find the index record with the largest key value less than or equal to the search key, and then follow the pointers sequentially.

Access Cost =log2(n)+1=log2(*n*)+1 *, where n is the number of blocks involved in the index file.*



Hash File Organization

Uses a hash function to map keys to buckets.

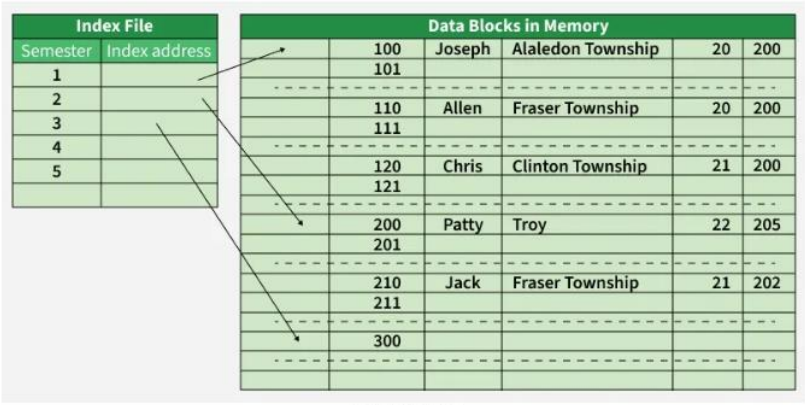
- Offers fast access for exact-match queries.
- Not suitable for range queries.

Types of Indexing Methods

There are different types of indexing techniques, each optimized for specific use cases.

1. Clustered Indexing

Clustered Indexing stores related records together in the same file, reducing search time and improving performance, especially for join operations. Data is stored in sorted order based on a key (often a non-primary key) to group similar records, like students by semester. If the indexed column isn't unique, multiple columns can be combined to form a unique key. This makes data retrieval faster by keeping related records close and allowing quicker access through the index.



Clustered Indexing

2. Primary Indexing

Primary indexing is an indexing technique in which the index is created on the primary key of a data file. The data records are physically stored in sorted order according to the primary key. The primary key uniquely identifies each record, each index entry corresponds to a block of records and contains the primary key value along with a pointer to the first record of that block. It is commonly used with sequential file organization and improves the efficiency of search operations because the data is ordered.

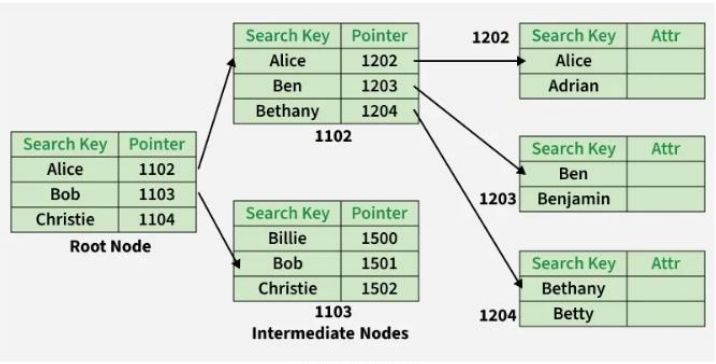
Key Features: The data is stored in sequential order, making searches faster and more efficient.

3. Non-clustered or Secondary Indexing

A non-clustered index just tells us where the data lies, i.e. it gives us a list of virtual pointers or references to the location where the data is actually stored. Data is not physically stored in the order of the index. Instead, data is present in leaf nodes.

Example: The contents page of a book. Each entry gives us the page number or location of the information stored. The actual data here(information on each page of the book) is not organized but we have an ordered reference(contents page) to where the data points actually lie. We can have only dense ordering in the non-clustered index as sparse ordering is not possible because data is not physically organized accordingly.

It requires more time as compared to the clustered index because some amount of extra work is done in order to extract the data by further following the pointer. In the case of a clustered index, data is directly present in front of the index.

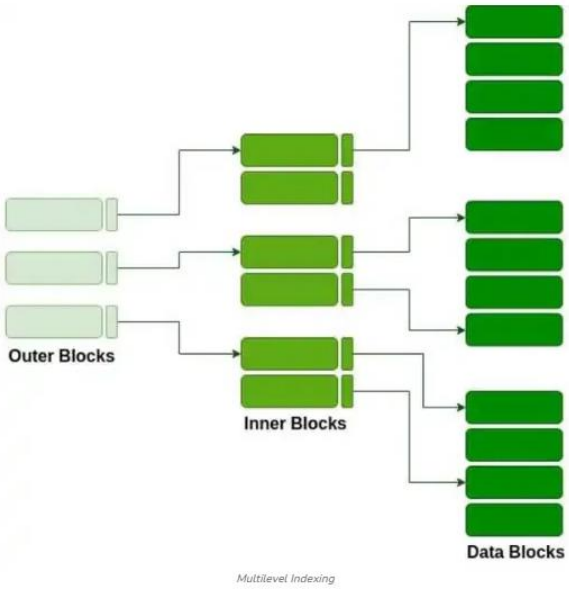


Non Clustered Indexing

4. Multilevel Indexing

With the growth of the size of the database, indices also grow. As the index is stored in the main memory, a single-level index might become too large a size to store with multiple disk accesses. The multilevel indexing segregates the main block into various smaller blocks so that the same can be stored in a single block.

The outer blocks are divided into inner blocks which in turn are pointed to the data blocks. This can be easily stored in the main memory with fewer overheads. This hierarchical approach reduces memory overhead and speeds up query execution.



Multilevel Indexing

Advantages

- **Faster Queries**: Indexes allow quick search of rows matching specific values, speeding up data retrieval.
- **Efficient Access**: Reduces disk I/O by keeping frequently accessed data in memory.
- **Improved Sorting**: Speeds up sorting by indexing the relevant columns.
- **Consistent Performance**: Maintains query speed even as data grows.
- **Data Integrity**: Ensures uniqueness in columns indexed as unique, preventing duplicate entries.

Disadvantages

- **Increased Storage Space**: Depending on the size of the data, this can significantly increase the overall storage requirements.
- **Increased Maintenance Overhead**: Indexes must be updated which can slow down these operations.
- **Slower Insert/Update Operations**: Indexes must be maintained and updated, inserting or updating data takes longer than in a non-indexed database.

- **Complexity in Choosing the Right Index**: Determining the appropriate indexing strategy for a particular dataset can be challenging and requires an understanding of query patterns and access behaviors.

Indexing: Case Study

Scenario: Library Book Lookup System

A college library has **100,000 books**. Students constantly search for books by **BookID**. Without any index, every search is slow. Let's see how indexing fixes this.

Table structure:

BOOK(BookID, Title, Author, Subject, Shelf)

Assumptions:

- Each block holds 10 records, so the file has **10,000 blocks**.
- The file is stored as a heap (unordered).

Step 1: Search Without an Index

Query: SELECT * FROM BOOK WHERE BookID = 7845;

The DBMS has no shortcut, so it scans block by block.

- Worst case: **10,000 block accesses**
- Average case: about **5,000 block accesses**

This is like finding a book by walking through every shelf in the library.

Step 2: Create an Index

sql

CREATE INDEX idx_bookid ON BOOK(BookID);

The index is a small, separate structure holding only **(BookID, pointer to block)** pairs, sorted by BookID.

Index (sample):

BookID	Block Pointer
1012	Block 45
3300	Block 812
5120	Block 97
7845	Block 2301
9001	Block 640

Analogy: This is the index at the back of a textbook. Instead of reading every page, you look up the term and jump straight to the page number.

Step 3: Search With the Index

Query: SELECT * FROM BOOK WHERE BookID = 7845;

1. Search the index and find BookID 7845, which points to Block 2301.
2. Go directly to Block 2301 and read the record.

Cost: about 3 index block reads (B+ tree height) + 1 data block read = **4 block accesses**.

Step 4: Comparison

Method	**Block Accesses**
No index (average)	~5,000
No index (worst)	10,000
With index	~4

The index makes the search roughly **1,000 times faster**.

Step 5: Range Query Benefit

Query: SELECT * FROM BOOK WHERE BookID BETWEEN 5000 AND 5100;

Because the index is sorted, the DBMS finds 5000 once and then reads forward until 5100. It never scans the whole file.

Step 6: The Cost of Indexing

Operation	**Effect of Index**
Search	Much faster
Insert	Slightly slower (index must also be updated)
Delete / Update	Slightly slower (index entry must change)
Storage	Extra space needed for the index

So indexes should be created only on columns that are searched often.

Types of Indexes (Quick Overview)

- **Primary index**: built on the sorted key of the file (e.g., BookID).
- **Secondary index**: built on a non-key column (e.g., Author or Subject).
- **Dense index**: one entry for every record.
- **Sparse index**: one entry per block, which uses less space.

Hash File Organization in DBMS

Hashing techniques are used to retrieve specific data. Searching through all index values to reach the desired data becomes very inefficient, in this scenario we may use hashing as an efficient technique

for locating desired data directly on disk without using an index structure. Hash File Configuration is also known as **Direct file configuration.** This permits for fast recovery of information based on a key

In Hashing we mainly refer the following terms:

- **Data Bucket:** A data bucket is a storage location where records are stored. These buckets are also considered storage units.
- **Hash Function:** A hash function is a mapping function that maps all search keys to actual record addresses. Generally, a hash function uses a primary key to generate a hash index (address of a data block). Hash functions range from simple to complex mathematical functions.
- **Hash Index:** The prefix of the entire hash value is used as the hash index. Each hash index has a depth value that indicates the number of bits used to calculate the hash function.

Hashing Technique

Data is stored in data blocks at addresses generated using a hash function. The location where these records are stored is called a data block or data bucket. In this organization, records are stored at known addresses rather than by location. To write a record, the address is first calculated by applying a mathematical function to obtain the record's key. The data record is saved to the generated address. In this case, the records are stored in BUCKETS, which are storage units that can store one or more records. For example, the hash function h(K) = K mod 7 hashes 35 and 43 to addresses 0, 1, as shown below,

43 mod 7 = 135 mod 7 = 0

Hashing Types

There are two types of hashing such as:

- Static Hashing
- Dynamic Hashing
- Open addressing
- Seperate chaining

1. Static Hashing

If you specify a search key value, the hash function always calculates the same address. If you want to generate an address that uses a mod 5 hash function, only 5 different values will be generated. The output address of this function is always the same. The number of available buckets always remains constant. Bucket addresses generated with static hashing always remain the same.

For example,

If you use the hash function mod(5) to get the address for customer ID =75, you will always get the same bucket address 0

The bucket address does not change in this scenario.

75 mod 5= 0

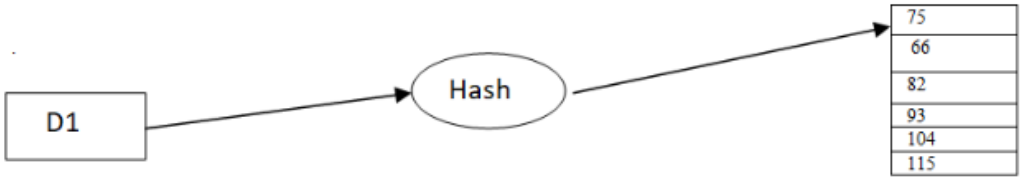
66 mod 5 = 1

82 mod 5 = 2

93 mod 5 =3

104 mod 5 = 4

and so on.



Static Hashing mapping with example

2. Dynamic Hashing

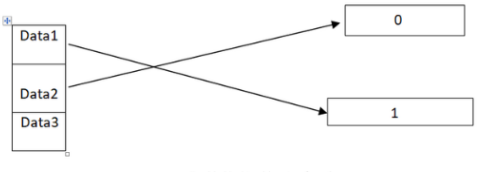
In dynamic hashing, Data buckets grow or shrink (dynamically added or removed) as the data set grows or shrinks. Dynamic Hashing is also known as Extended Hashing. Dynamic hashing requires the hash function to generate a large number of values.

For example, there are three data sets: Data1, Data2, and Data3.

The hash function produces three addresses 1010, 1011, and 1001. This storage method only considers part of this address, specifically the first bit that stores the data.

So we try to load three of them into addresses 0 and 1.

h(Data 1) -> 1010
h(Data 2) -> 1011
h(Data 3) -> 1001

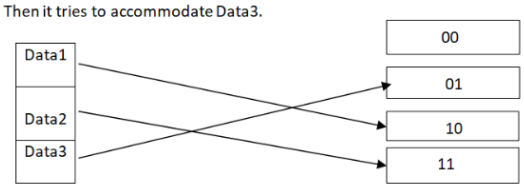


Double Hashing Mapping Case 1

But the problem is that there are no bucket addresses left for Data3. Buckets must be dynamically expanded to support D3. Therefore, we change the address by 2 bits instead of 1 bit and update the existing data to have a 2-bit address.

Next, try to record data 3.

Then it tries to accommodate Data3.



Double Hashing Mapping Case -2

Clustered File Organization in DBMS

Clustered File Organization stores related records from different tables together based on a common field, like a foreign key. This improves query performance for join operations by placing related data physically close on disk.

Key Terminologies

- **File Organization**: The technique that is used to store data in a file. Thus, file arrangement affects the efficiency of data access and retrieval.
- **Clustered index**: The arrangement of the physical data in rows is dictated by an index referred to as a clustered index. The key for the clustered index referred to as the **cluster key** is used to order the table.
- **Cluster Key:** A shared field from which related records from various tables are combined. Usually, a foreign key from one table and a primary key from another are involved.
- **Indexing**: Database tables can get data faster with the help of the technique involving the construction of a data structure that is referred to as indexing.
- **Primary Key**: A key that can uniquely identify a record in a table is known as the primary key.

A clustered file organization keeps two or more related tables/records in a single file known as a cluster. These files consist of two or more tables within a single data block, and the mapping attributes, defining the relationships between the tables, are stored only once. These contain two or more tables in one data block and the key attributes that are related between the tables are stored only once.

This means it is cheaper to search for and retrieve distinct records from different files as they are now integrated and stored in the same cluster. Let us understand the concept better with the following examples.

Examples

Example 1: let's consider a database for an online store with two tables: "Customers" and "Orders" .

Customer table:

customer_id	customer_name	address
1	Ramesh Sharma	Delhi
2	Priya Patel	Lucknow
3	Sanjay Gupta	Patna

Orders table:

order_id	customer_id	order_date	amount
101	1	2024-05-01	Rs. 1500
102	2	2024-05-03	Rs. 10000
103	1	2024-05-04	Rs. 2500
104	3	2024-05-06	Rs. 500

The system must quickly combine these databases in order to obtain the necessary data when a consumer requests information about their purchasing history. Clustered file organization can greatly improve this procedure.

By clustering the tables on the "customer_id"**(cluster key**), the records would be grouped as follows:

customer_id	customer_name	address	order_id	order_date	amount
1	Ramesh Sharma	Delhi	101	2024-05-01	Rs. 1500
1	Ramesh Sharma	Delhi	103	2024-05-04	Rs. 2500
2	Priya Patel	Lucknow	102	2024-05-03	Rs. 10000
3	Sanjay Gupta	Patna	104	2024-05-06	Rs. 500

Because of this organization, the database may quickly retrieve any relevant information in a single, contiguous block of storage when a query is made for Ramesh Sharma's order history, eliminating the need for numerous I/O operations.

Types of Clustered file organization

1. Indexed Clusters

The clusters are organized, such that records in an indexed cluster are stored in the order of the clustering key. However, for searching and retrieving data, an index is created on the clustering key along with physical sorting as well. When using the range queries and equality searches on the clustering key, this kind of clustering is beneficial.

2. Hash Clusters

In a hash cluster, every record is located in accordance with a hash function on the clustering key. By applying it, one can identify records with the same hash value, and therefore identify their physical location. Another great advantage of hash clustering is that by using their clustering key, it is possible to obtain an individual record very quickly. Still, since records are not sorted in a particular order, the algorithm is slower for range queries.

Transaction in DBMS

A transaction refers to a sequence of one or more operations (such as read, write, update, or delete) performed on the database as a single logical unit of work.

- A transaction ensures all operations are completed or none are applied (commit or rollback).
- It maintains database integrity, consistency, and reliability.

All types of database access operation which are held between the beginning and end transaction statements are considered as a single logical transaction. During the transaction the database is inconsistent. Only once the database is committed the state is changed from one consistent state to another.

Example: Let's consider an online banking applicatio**n**:

Transaction: When a user performs a money transfer, several operations occur, such as:

- **Reading** the account balance of the sender.
- **Writing** the deducted amount from the sender's account.
- **Writing** the added amount to the recipient's account.

Operations of Transaction

A user can make different types of requests to access and modify the contents of a database. So, we have different types of operations relating to a transaction. They are discussed as follows:

1) Read(X)

A read operation is used to read the value of a particular database element **X** and stores it in a temporary buffer in the main memory for further actions such as displaying that value.

Example: For a banking system, when a user checks their balance, a Read operation is performed on their account balance:

SELECT balance FROM accounts WHERE account_id = 'A123';

This retrieves the user's current account balance. It does not modify or update the database.

2) Write(X)

A write operation stores updated data from main memory back to the database. It usually follows a read, where data is fetched, modified (e.g., arithmetic changes) and then written back to save the updated value.

Example: For the banking system, if a user withdraws money, a Write operation is performed after the balance is updated:

UPDATE accounts SET balance = balance - 100 WHERE account_id = 'A123';

This updates the balance of the user's account after withdrawal.

3) Commit

This operation in transactions is used to maintain integrity in the database. Due to some failure of power, hardware, or software, etc., a transaction might get interrupted before all its operations are completed. This may cause ambiguity in the database, i.e. it might get inconsistent before and after the transaction.

Example: After a successful money transfer in a banking system, a Commit operation finalizes the transaction:

COMMIT;

Once the transaction is committed, the changes to the database are permanent and the transaction is considered successful.

4) Rollback

A rollback undoes all changes made by a transaction if an error occurs, restoring the database to its last consistent state. It helps prevent data inconsistency and ensures safety.

Example: Suppose during the money transfer process, the system encounters an issue, like insufficient funds in the sender's account. In that case, the transaction is rolled back:

ROLLBACK;

This will undo all the operations performed so far and ensure that the database remains consistent.

ACID Properties of Transaction

Transactions in DBMS must ensure data is accurate and reliable. They follow four key ACID properties:

- **Atomicity:** A transaction is fully completed or fully rolled back.
- **Consistency:** Maintains the database in a valid state after a transaction.
- **Isolation:** Transactions execute independently without interfering with each other.
- **Durability:** Committed changes remain permanent, even after a system failure.

Transaction Schedules

When multiple transaction requests are made at the same time, we need to decide their order of execution. Thus, a transaction schedule can be defined as a chronological order of execution of multiple transactions.

Example: After a successful transfer, the updated balance remains safe despite a power failure.

There are broadly two types of transaction schedules discussed as follows:

1. **Serial Schedule:** Transactions execute one after another, ensuring consistency but reducing performance.
2. **Non-Serial Schedule:** Transactions execute concurrently, improving efficiency and requiring concurrency control.

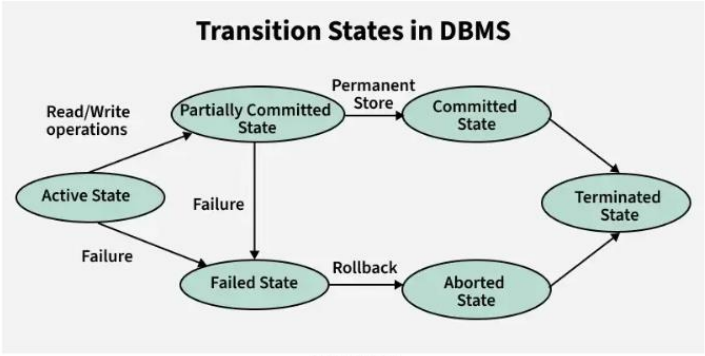
Transaction States

Transaction states represent the different phases a transaction goes through during its lifecycle in a database system. These states help track the progress and outcome of a transaction to ensure data consistency and integrity.

Different Types of Transaction States in DBMS

These are the different types of Transaction States:

1. Active State
2. Partially Committed State
3. Committed State
4. Failed State
5. Aborted State
6. Terminated State



Transaction States

1. Active State

- It is the first stage of any transaction when it has begun to execute. The execution of the transaction takes place in this state.
- Operations such as insertion, deletion, or updation are performed during this state.
- During this state, the data records are under manipulation and they are not saved to the database, rather they remain somewhere in a buffer in the main memory.

2. Partially Committed

- The transaction has finished its final operation, but the changes are still not saved to the database.
- After completing all read and write operations, the modifications are initially stored in main memory or a local buffer. If the changes are made permanent in the database then the state will change to "committed state" and in case of failure it will go to the "failed state".

3. Committed

This state of transaction is achieved when all the transaction-related operations have been executed successfully along with the Commit operation, i.e. data is saved into the database after the required manipulations in this state. This marks the successful completion of a transaction.

4. Failed State

If any of the transaction-related operations cause an error during the active or partially committed state, further execution of the transaction is stopped and it is brought into a failed state. Here, the database recovery system makes sure that the database is in a consistent state.

5. Aborted State

If a transaction reaches the failed state due to a failed check, the database recovery system will attempt to restore it to a consistent state. If recovery is not possible, the transaction is either rolled back or cancelled to ensure the database remains consistent.

In the aborted state, the DBMS recovery system performs one of two actions:

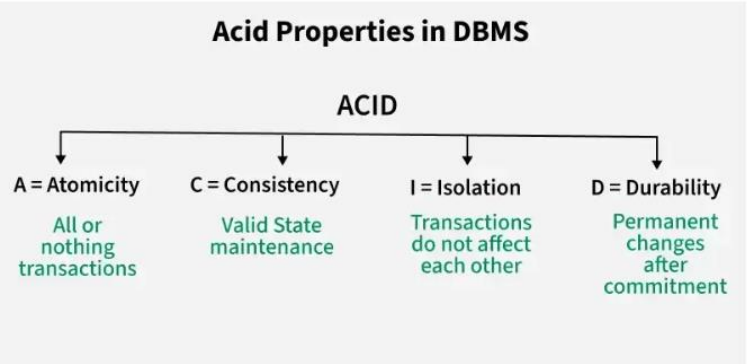
- **Kill the transaction**: The system terminates the transaction to prevent it from affecting other operations.
- **Restart the transaction**: After making necessary adjustments, the system reverts the transaction to an active state and attempts to continue its execution.

6. Terminated State

It refers to the final state of a transaction, indicating that it has completed its execution. Once a transaction reaches this state, it has either been successfully committed or aborted. In this state, no further actions are required from the transaction, as the database is now stable.

ACID Properties

Transactions are fundamental operations that allow us to modify and retrieve data. However, to ensure the integrity of a database, it is important that these transactions are executed in a way that maintains consistency, correctness, and reliability even in the case of failures/errors. This is where the ACID properties come into play.



ACID Properties: A Practical Example

I'll use a classic scenario: **transferring ₹5,000 from Account A to Account B** in a banking system.

Setup

CREATE TABLE accounts (

```
    account_id INT PRIMARY KEY,
    holder     VARCHAR(50) NOT NULL,
    balance    DECIMAL(12,2) NOT NULL CHECK (balance >= 0)
);
INSERT INTO accounts VALUES (1, 'Asha',  10000.00);
INSERT INTO accounts VALUES (2, 'Rahul', 3000.00);
```

The Transaction

```
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 5000 WHERE account_id = 2;
COMMIT;
```

Now let's see how each ACID property applies.

A: Atomicity (all or nothing)

The transfer is two operations: debit and credit. Either both happen or neither does.

Failure scenario: Asha tries to send ₹15,000 (she only has ₹10,000).

```
BEGIN;
UPDATE accounts SET balance = balance - 15000 WHERE account_id = 1;
-- ERROR: violates CHECK (balance >= 0)
ROLLBACK;  -- automatic on error
```

The credit to Rahul never happens, and any partial work is undone. Without atomicity, money could vanish: debited from one account but never credited to the other.

Explicit control in application code (Python):

```
import psycopg2
conn = psycopg2.connect("dbname=bank user=postgres")
try:
    with conn:                  # commits on success, rolls back on exception
        with conn.cursor() as cur:
            cur.execute("UPDATE accounts SET balance = balance - %s WHERE account_id = %s", (5000, 1))
            cur.execute("UPDATE accounts SET balance = balance + %s WHERE account_id = %s", (5000,
2))
except Exception as e:
    print("Transfer failed, rolled back:", e)
```

C: Consistency (valid state to valid state)

The database must always satisfy its rules. Here those rules are:

- Balance can never be negative (CHECK constraint)
- Total money in the system is unchanged by a transfer

-- Before: 10000 + 3000 = 13000

-- After : 5000 + 8000 = 13000 ✓

SELECT SUM(balance) FROM accounts;

If a transaction would break a constraint, the database rejects it, so it can never be committed in an invalid state. You can add further rules with foreign keys, UNIQUE, and triggers.

I: Isolation (concurrent transactions don't interfere)

Problem scenario: two transactions run at the same time on Asha's account (balance ₹10,000).

Time	Transaction T1 (withdraw 8000)	Transaction T2 (withdraw 8000)
t1	reads balance = 10000	
t2		reads balance = 10000
t3	writes 10000 - 8000 = 2000	
t4		writes 10000 - 8000 = 2000

Both succeed, but ₹16,000 was withdrawn from a ₹10,000 account. This is a **lost update**.

Solution: row-level locking

BEGIN;

SELECT balance FROM accounts WHERE account_id = 1 FOR UPDATE; -- locks the row

UPDATE accounts SET balance = balance - 8000 WHERE account_id = 1;

COMMIT;

T2 now has to wait until T1 commits, then sees the balance of ₹2,000, and its withdrawal fails the CHECK constraint.

Isolation levels let you tune the trade-off between safety and speed:

Level	Dirty Read	Non-repeatable Read	Phantom Read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible

Level	Dirty Read	Non-repeatable Read	Phantom Read
REPEATABLE READ	Prevented	Prevented	Possible*
SERIALIZABLE	Prevented	Prevented	Prevented

*In PostgreSQL, REPEATABLE READ also prevents phantoms.

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- ... statements ...

COMMIT;
```

D: Durability (committed data survives crashes)

Once COMMIT returns successfully, the transfer is permanent, even if the server loses power one millisecond later.

How databases achieve this:

1. **Write-Ahead Log (WAL):** changes are first written to a sequential log on disk and flushed (fsync) *before* COMMIT is acknowledged.
2. On restart after a crash, the database **replays the log** to recover all committed transactions and discards uncommitted ones.

```
Client: COMMIT
  |
  ▼
[Write to WAL → fsync to disk] → "COMMIT OK" returned to client
  |
  ▼
(Data files updated later in the background)
```

If the crash happens **before** the WAL flush, the transaction is rolled back (consistent with atomicity). If it happens **after**, the transaction is recovered from the log.

Full Example: Safe Transfer Function

```
CREATE OR REPLACE FUNCTION transfer_money(from_id INT, to_id INT, amount DECIMAL)
RETURNS VOID AS $$
BEGIN
  -- Lock rows in a consistent order to avoid deadlocks
  PERFORM 1 FROM accounts
  WHERE account_id IN (from_id, to_id)
  ORDER BY account_id
```

```
    FOR UPDATE;
    UPDATE accounts SET balance = balance - amount WHERE account_id = from_id;
    UPDATE accounts SET balance = balance + amount WHERE account_id = to_id;
    -- CHECK constraint raises an error automatically if funds are insufficient
END;
$$ LANGUAGE plpgsql;
-- Usage (the function runs inside a transaction)
BEGIN;
SELECT transfer_money(1, 2, 5000);
COMMIT;
```

Serializability

In DBMS, serializability ensures the database remains correct even when multiple transactions run at the same time. It makes transactions behave as if they were executed one after another in some order. This prevents conflicts, maintains data consistency, and ensures reliable results, just like in a single, orderly sequence of transactions.

Non-serial Schedule

A non-serial schedule allows transactions to run concurrently and may access the same data. To ensure database consistency, it must be serializable, meaning it should produce the same result as some serial (one-by-one) execution.

Example:

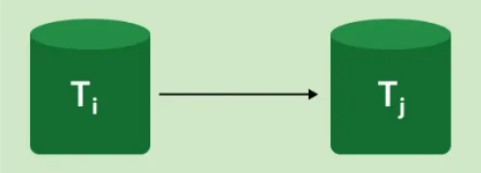
Transaction-1	Transaction-2
R(a)	
W(a)	
	R(b)
	W(b)
R(b)	

Transaction-1	Transaction-2
	R(a)
W(b)	
	W(a)

We can observe that Transaction-2 begins its execution before Transaction-1 is finished, and they are both working on the same data, i.e., "a" and "b", interchangeably. Where "R"-Read, "W"-Write

Serializability testing

We can utilize the Serialization Graph or Precedence Graph to examine a schedule's serializability. A schedule's full transactions are organized into a Directed Graph, what a serialization graph is

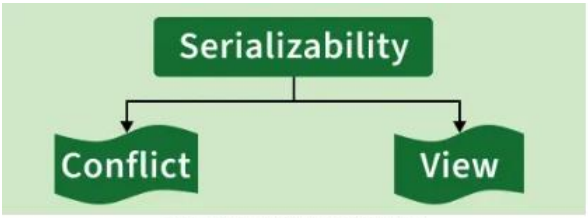


Precedence Graph

It can be described as a Graph G(V, E) with vertices V = "V1, V2, V3,..., Vn" and directed edges E = "E1, E2, E3,..., En". One of the two operations—READ or WRITE—performed by a certain transaction is contained in the collection of edges. Where Ti -> Tj, means Transaction-Ti is either performing read or write before the transaction-Tj.

Types of Serializability

There are two ways to check whether any non-serial schedule is serializable.



Types of Serializability - Conflict & View

1. Conflict serializability

Conflict serializability ensures database consistency by checking if a non-serial schedule can be rearranged into a serial order by swapping non-conflicting operations. It prevents conflicting operations (like read/write on the same data) from executing at the same time.

Conflict Equivalency

For two schedules (S1 and S2) to be conflict equivalent, they must satisfy the following:

- **Same Conflicting Operations:** The same conflicting operations (e.g., reads and writes on the same data item) must occur in both schedules in the same order. Example: If t1 writes A before t2 in S1, then t1 must also write A before t2 in S2.
- **Non-Conflicting Operations:** Operations that do not conflict (e.g., reading different data items) should not affect the order of the schedules. Example: If t1 reads B and t2 reads C in S1, then t1 should still read B and t2 should still read C in S2.
- **Preserving Transaction Order:** The order of conflicting operations should be the same in both schedules to maintain consistency. Example: If t1 writes A and t2 reads A in S1, t2 must read A after t1 in S2.

In simple terms, conflict equivalency ensures that conflicting operations happen in the same order in both schedules, while non-conflicting ones can appear in any order.

2. View Serializability

View serializability ensures that a non-serial schedule results in the same final outcome as a serial schedule, maintaining database consistency.

To further understand view serializability in DBMS, we need to understand the schedules S1 and S2. The two transactions T1 and T2 should be used to establish these two schedules. Each schedule must follow the three transactions in order to retain the equivalent of the transaction. These three circumstances are listed below.

1. **Same Transactions**: Both schedules must include the same set of transactions.
2. **Same Writes**: Each data item must be written by the same transaction in both schedules.
3. **Same Reads**: Each read must read the same value (from the same write) in both schedules.

View Equivalency

Schedules (S1 and S2) must satisfy these two requirements in order to be view equivalent:

1. The same data must be read first in both schedules. Example: If t1 reads A in S1, it must do the same in S2.
2. The same data must be used for the final write. Example: If t1 updates A last in S1, it should do the same in S2.
3. The middle sequence should also match. Example: If t1 reads A and t2 updates A in S1, the same order should occur in S2.

Example: We have a schedule "S" with two concurrently running transactions, "t1" and "t2."

Schedule S:

Transaction-1 (t1)	Transaction-2 (t2)
R(a)	

Transaction-1 (t1)	Transaction-2 (t2)
W(a)	
	R(a)
R(b) W(b)	W(a)
	R(b) W(b)

By switching between both transactions' mid-read-write operations, let's create its view equivalent schedule (S').

Schedule S':

Transaction-1 (t1)	Transaction-2 (t2)
R(a)	
W(a) R(b) W(b)	
	R(a)

Transaction-1 (t1)	Transaction-2 (t2)
	W(a)
	R(b)
	W(b)

It is a view serializable schedule since a view similar schedule is conceivable.

***Note**: A conflict serializable schedule is always viewed as serializable, but vice versa is not always true.*

Benefits of Serializability

- **Predictable Execution**: Transactions behave consistently without surprises—no data loss or corruption.
- **Easier Debugging**: Each transaction runs independently, making errors easier to trace and fix.
- **Lower Costs**: Reduces need for complex hardware and can cut down development costs.
- **Better Performance**: Optimized, consistent execution can sometimes outperform non-serial schedules.

Serializability vs Related Concepts

Definition

- Serializability: A concurrent schedule is serializable if its result is equivalent to some serial schedule (transactions executed one after another). It is the formal criterion for correct concurrent execution.
- Serializability: A concurrent schedule is serializable if its result is equivalent to some serial schedule (transactions executed one after another). It is the formal criterion for correct concurrent execution.

Differences

Concept	Meaning	Difference from Serializability
Serial Schedule	Transactions run one at a time, no interleaving	Serial is always correct but slow. Serializable allows interleaving yet gives the same result as a serial one.
Recoverability	A transaction commits only after the transaction it read from commits	Serializability ensures a correct result; recoverability ensures safe **rollback** after an abort. A schedule can be serializable but not recoverable.

Concept	Meaning	Difference from Serializability
Isolation (ACID)	Degree to which concurrent transactions are hidden from each other	Isolation is the *property*; serializability is its strongest *formal definition*. Lower isolation levels relax it.
Concurrency Control (2PL, Timestamp, MVCC)	Protocols that manage concurrent access	Serializability is the *goal*; concurrency control is the *mechanism* that achieves it.
Consistency (ACID)	Database constraints are preserved	Consistency concerns *data rules*; serializability concerns *correct interleaving*.

- **Conflict vs View Serializability**

Conflict Serializability	View Serializability
Based on order of conflicting operations	Based on initial reads, read-from, and final writes
Tested with a precedence graph (no cycle means serializable)	Testing is NP-complete
Subset of view serializable	Superset
Used in practice	Rarely used

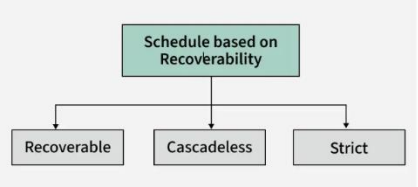
Recoverability in DBMS

Recoverability ensures that after a failure, the database can restore a consistent state by keeping committed changes and undoing uncommitted ones. It uses logs to redo or undo actions, preventing data loss and maintaining integrity.

There are several levels of recoverability that can be supported by a database system:

- **No-Undo Logging**: Only saves committed transactions; can't undo uncommitted ones.
- **Undo Logging**: Can undo uncommitted transactions but may lose some committed updates.
- **Redo Logging**: Can redo committed transactions to ensure durability.
- **Undo-Redo Logging**: Supports both undo and redo, ensuring full recovery and consistency.

Overall, recoverability is a crucial property of database systems, as it ensures that data is consistent and durable even in the event of failures or errors. It is important for database administrators to understand the level of recoverability provided by their system and to configure it appropriately to meet their application's requirements.



Recoverable Schedules

A recoverable schedule ensures a transaction commits only after the transactions it depends on have committed. This avoids inconsistencies and helps the database recover correctly after failures.

Example 1:

Consider the following schedule involving two transactions T_1 and T_2.

T_1	T_2
R(A)	
W(A)	
	W(A)
	R(A)
commit	
	commit

This is a recoverable schedule since T_1 commits before T_2, that makes the value read by T_2 correct.

Example 2:

S1: R1(x), W1(x), R2(x), R1(y), R2(y),W2(x), W1(y), C1, C2;

T1	T2
R (x)	
W (x)	
	R (x)
R (y)	
	R (y)
	W (x)
W (y)	
Commit	
	Commit

- T2 reads uncommitted data from T1 (R2(x) depends on W1(x)), but T2 commits only after T1 commits.
- The schedule is recoverable.

Concurrency Issues and Their Importance in DBMS

1. Introduction

Concurrency means multiple transactions executing at the same time on shared data. It improves performance, but if their operations interleave without control, the database can become incorrect. The problems that arise are called **concurrency issues** (or concurrency anomalies).

2. Types of Concurrency Issues

A. Lost Update (Write-Write Conflict)

Two transactions read the same value and both overwrite it, so one update is lost.

Initial: A = 100

T1: R(A) → 100

T2: R(A) → 100

T1: A = A + 50; W(A) → 150

T2: A = A + 20; W(A) → 120 ← T1's update lost!

Correct result is 170, but the database holds 120.

B. Dirty Read (Write-Read Conflict)

A transaction reads data written by another transaction that has **not yet committed**.

T1: W(A) (A: 100 → 500)

T2: R(A) → 500 ← uncommitted value

T1: ABORT ← A returns to 100

T2 used a value that never officially existed.

C. Non-Repeatable Read (Read-Write Conflict)

A transaction reads the same item twice and gets **different values**, because another transaction updated and committed in between.

T1: R(A) → 100

T2: W(A) = 200; COMMIT

T1: R(A) → 200 ← different result in same transaction

D. Phantom Read

A transaction re-runs a query and finds **new or missing rows**, because another transaction inserted or deleted rows matching the condition.

T1: SELECT * FROM emp WHERE dept = 'IT' → 5 rows

T2: INSERT INTO emp (dept = 'IT'); COMMIT

T1: SELECT * FROM emp WHERE dept = 'IT' → 6 rows ← phantom

E. Incorrect Summary (Inconsistent Analysis)

An aggregate computed while another transaction is updating the data gives a wrong total.

T1: transfer 50 from A to B (A = 100, B = 200)

T2: SUM(A + B) reads A after debit but B before credit → 250 (true total 300)

F. Deadlock and Starvation (Side-effects of Locking)

Issue	Description
Deadlock	T1 waits for a lock held by T2, while T2 waits for a lock held by T1; neither can proceed
Starvation	A transaction waits indefinitely because others keep getting the lock

Summary of Issues and Solutions

Issue	Conflict Type	Prevented By
Lost update	W-W	Exclusive locks, SELECT ... FOR UPDATE
Dirty read	W-R	READ COMMITTED or higher
Non-repeatable read	R-W	REPEATABLE READ or higher
Phantom read	Range R-W	SERIALIZABLE, predicate/range locks
Incorrect summary	R-W	Serializable schedules, snapshots
Deadlock	Lock cycle	Deadlock detection, timeouts, ordered locking

Importance of Concurrency Control in DBMS

Concurrency itself is essential, and controlling it is equally essential.

Why concurrency is needed:

1. **Higher throughput:** many transactions are processed per second.
2. **Better resource utilisation:** while one transaction waits for disk I/O, the CPU serves another.
3. **Reduced waiting/response time:** short transactions need not wait behind long ones.
4. **Multi-user support:** thousands of users (banking, e-commerce, reservations) access data simultaneously.

Failure Recovery in a System

1. Introduction

Failure recovery is the set of techniques that restores a system to a correct, consistent state after a failure (hardware crash, software error, power loss, network fault or transaction abort), so that no data is lost and the system can resume normal operation. A failure is a deviation from expected behaviour; recovery is the process of detecting it and bringing the system back.

2. Types of Failures

Type	Example
Transaction failure	Logical error, deadlock, abort
System failure	OS crash, power cut (memory lost, disk intact)
Media failure	Disk head crash, corrupted storage
Communication failure	Network partition, lost messages (distributed systems)

3. Steps in Failure Recovery

1. **Failure detection**: using timeouts, heartbeats, exception handling and error codes.
2. **Damage assessment**: identify which processes or transactions were affected.
3. **Recovery action**: restore the last consistent state using one of the techniques below.
4. **Restart and resume**: continue operation, with the redo/undo of pending work completed.

4. Implementation Techniques

a) Backup and Restore

Periodic full and incremental backups are stored on separate media. After a media failure, the latest backup is restored and then updated using archived logs. This is simple but can lose data created after the last backup.

b) Logging (Write-Ahead Logging, WAL)

Every change is first written to a **log** on stable storage *before* the database is modified. A log record holds: <Transaction ID, Data item, Old value, New value>.

- **UNDO**: reverses changes of uncommitted transactions.
- **REDO**: reapplies changes of committed transactions that never reached disk.

c) Checkpointing

The system periodically writes all buffered updates to disk and records a **checkpoint** in the log. After a crash, recovery only needs to examine the log from the last checkpoint rather than from the beginning, which makes recovery faster.

d) Shadow Paging

Two page tables are maintained: a *current* page table and a *shadow* page table. Updates go to new pages; on commit the current table becomes the shadow. After a crash the shadow table is used, so no undo/redo is required.

e) Redundancy and Replication

Components are duplicated (RAID, standby servers, database replication, clustering). When the primary fails, a **failover** switches to the replica, keeping downtime minimal.

f) Retry, Rollback and Checkpoint-Restart

Transient errors are handled by retrying the operation. Long-running programs save their state periodically and restart from the last saved state.

5. Illustration: Recovery Using Log and Checkpoint

Time ——————————————————————————————►

T1: start —— commit

checkpoint

T2: start ———————— commit ✕ CRASH

T3: start ———————————— (uncommitted)

Recovery procedure after the crash:

- **T1** committed before the checkpoint, so no action is needed.
- **T2** committed after the checkpoint, so it is **REDONE**.
- **T3** had not committed, so it is **UNDONE** (rolled back).

The database is therefore restored to a consistent state in which only committed transactions have effects, preserving atomicity and durability (the A and D of ACID).

6. Example Scenario (Online Banking)

A transfer of ₹5,000 from account A to B fails after debiting A but before crediting B.

- The log shows the transaction as uncommitted.
- Recovery performs **UNDO**, restoring A's balance.
- The customer can retry, and money is never lost or duplicated.

7. Distributed Systems Considerations

- **Two-Phase Commit (2PC)** ensures all nodes commit or all abort.
- **Heartbeat monitoring** detects crashed nodes.
- **Leader election** picks a new coordinator on failure.

Concurrency Control in DBMS

In a Database Management System (DBMS), Concurrency control is a mechanism in DBMS that allows simultaneous execution of transactions while maintaining ACID properties - Atomicity, Consistency, Isolation and Durability. It maintains the integrity, accuracy and reliability of data when multiple users or processes perform read/write operations concurrently. It helps manage issues like:

- Conflicting operations on shared data
- Inconsistent database states
- Lost or incorrect updates

Need of Concurrency Control

Concurrency control is essential to:

Scenario	Without Concurrency Control	With Concurrency Control
Initial Balance	$1000	$1000
Transaction A (User 1)	Withdraws $200: Reads balance as $1000, calculates new balance $800.	Withdraws $200: Reads balance as $1000, calculates new balance $800.
Transaction B (User 2)	Withdraws $300: Reads balance as $1000 (before A updates), calculates $ 700	Waits for Transaction A to finish. Once A commits, reads updated balance $800, calculates new balance $500.
Final Updates	A writes $800. Then, B reads overwrites with $700 (lost update issue).	A writes $800. Then, B reads updated balance and writes $500 (consistent result).
Final Balance	$700 (incorrect, inconsistent).	$500 (correct, consistent).

Concurrency Control

- Prevent conflicts between simultaneous transactions.
- Maintain data consistency and accuracy in multi-user environments.
- Avoid problems such as dirty reads, lost updates and inconsistent reads.

Example:

- Without concurrency control: Two users update the same record simultaneously and one update overwrites the other.
- With concurrency control: The DBMS uses locks or timestamps to ensure updates occur sequentially and data remains correct.

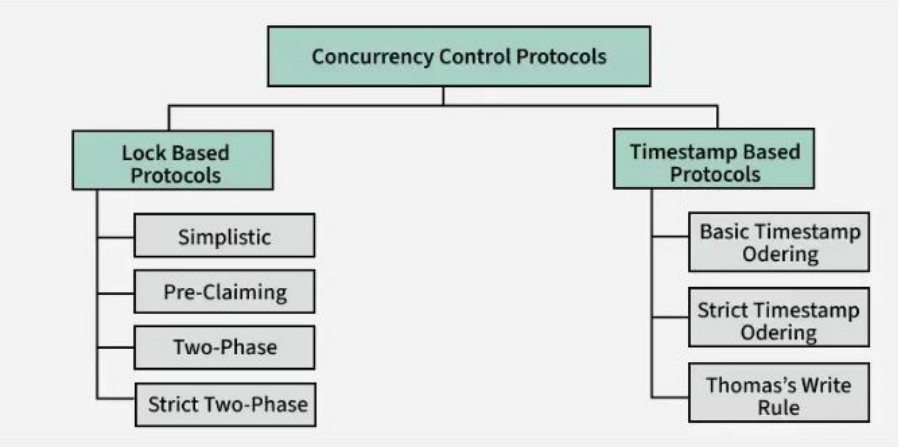
Concurrency Problems in DBMS

When multiple transactions execute concurrently, several problems may occur:

- Dirty Read: A transaction reads uncommitted data from another transaction that may later roll back.
- Lost Update: Two transactions update the same data and one update overwrites the other.
- Inconsistent Read: A transaction reads the same data multiple times and the data changes in between reads.

Concurrency Control Protocols

Concurrency control protocols define rules to ensure correct and consistent execution of transactions. The main protocols are:



Concurrency Control Protocols

1. Lock-Based Concurrency Control:

- Uses locks to restrict access to data items during a transaction. Common types include shared locks (read) and exclusive locks (write).
- Ensures serializability and prevents conflicts.
- Example: Two-Phase Locking (2PL) guarantees that once a transaction releases a lock, it cannot obtain any new locks.

2. Timestamp-Based Concurrency Control:

- Each transaction is assigned a timestamp.
- The DBMS uses these timestamps to order transactions and prevent conflicts based on their start time.

Recoverable and Cascadeless Schedules

Concurrency control also ensures proper scheduling of transactions:

Recoverable Schedules:

- A transaction commits only if all transactions it depends on have committed.
- Prevents inconsistencies caused by dependent transactions.
- Example: Strict 2PL ensures recoverability by delaying commits until all dependent transactions commit.

Cascadeless Schedules:

- Avoids cascading rollbacks, where one failed transaction causes others to fail.
- Achieved by allowing transactions to read data only after the previous transaction commits.

Advantages of Concurrency Control

1. Reduced Waiting Time: Multiple transactions can proceed simultaneously, reducing idle time.
2. Improved Response Time: Faster access and interaction with the database.
3. Better Resource Utilization: Hardware and database resources are efficiently shared.

4. Increased System Efficiency: Higher throughput and better overall performance.

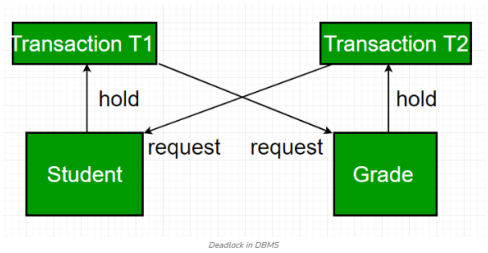
Disadvantages of Concurrency Control

1. Overhead: Managing locks and timestamps adds system overhead.
2. Deadlocks: Circular waits between transactions can halt progress.
3. Reduced Concurrency: Locking can limit the number of simultaneous transactions.
4. Complexity: Implementation in distributed or large systems can be difficult.
5. Inconsistencies: Rollbacks or long waits may cause temporary data inaccuracy or staleness.

Deadlock in DBMS

A deadlock occurs in a multi-user database environment when two or more transactions block each other indefinitely by each holding a resource the other needs. This results in a cycle of dependencies (circular wait) where no transaction can proceed.

For Example: Consider the image below



- T1 locks Resource "Student" and needs Resource "Grade"
- T2 locks Resource "Grade" and needs Resource "Student"
- T1 waits for T2, T2 waits for T1, hence resulting in a deadlock

Necessary Conditions of Deadlock

For a deadlock to occur, all four of these conditions must be true:

- Mutual Exclusion: Only one transaction can hold a particular resource at a time.
- Hold and Wait: The Transactions holding resources may request additional resources held by others.
- No Preemption: The Resources cannot be forcibly taken from the transaction holding them.
- Circular Wait: A cycle of transactions exists where each transaction is waiting for the resource held by the next transaction in the cycle.

Why Deadlocks Are a Problem?

- Transactions are stuck indefinitely.
- System throughput decreases as transactions remain blocked.
- Resources are held unnecessarily, preventing other operations.
- Can lead to performance bottlenecks or even system-wide standstill if not handled.

Real-Life Example

Transaction T1:

- Locks rows in Students
- Wants to update rows in Grades

Transaction T2:

- Locks rows in Grades
- Wants to update rows in Students

Both wait on each other and this results in deadlock, and all database activity comes to a standstill.

How to Handle Deadlocks

There are some approaches and by ensuring them, we can handle deadlocks. They are discussed below:

1. Deadlock Avoidance

Plan transactions in a way that prevents deadlock from occurring.

Methods:

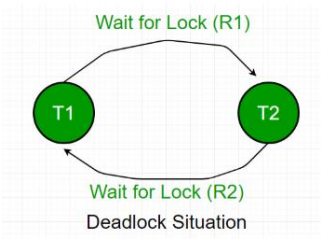
- Access resources in the same order. For e.g., always access Students first, then Grades
- Use row-level locking and READ COMMITTED isolation level. It reduces chances, but doesn't eliminate deadlocks.completely

2. Deadlock Detection

If a transaction waits too long, the DBMS checks if it's part of a deadlock.

Method: Wait-For Graph

- Nodes: Transactions
- Edges: Waiting relationships
- If there's a cycle, a deadlock exists. It's mostly suitable for small to medium databases



3. Deadlock Prevention

For a large database, the deadlock prevention method is suitable. A deadlock can be prevented if the resources are allocated in such a way that a deadlock never occurs. The DBMS analyzes the operations whether they can create a deadlock situation or not, If they do, that transaction is never allowed to be executed.

Deadlock prevention mechanism proposes two schemes:

1. Wait-Die Scheme (Non-preemptive)

- Older transactions are allowed to wait.
- Younger transactions are killed (aborted and restarted) if they request a resource held by an older one

For example:

- Consider two transaction- T1 = 10 and T2 = 20
- If T1 (older) wants a resource held by T2 → T1 waits
- If T2 (younger) wants a resource held by T1 → T2 dies and restarts

Prevents deadlock by not allowing a younger transaction to wait and form a wait cycle.

2. Wound-Wait Scheme (Preemptive)

- Older transactions are aggressive (preemptive) and can force younger ones to abort.
- Younger transactions must wait if they want a resource held by an older one.

For example:

- Consider two transaction- T1 = 10 and T2 = 20
- If T1 (older) wants a resource held by T2 → T2 is killed, T1 proceeds.
- If T2 (younger) wants a resource held by T1 → T2 waits

Prevents deadlock by not allowing younger transactions to block older ones.

Timestamp based Concurrency Control

Timestamp-based concurrency control is a technique used in database management systems (DBMS) to ensure serializability of transactions without using locks. It uses timestamps to determine the order of transaction execution and ensures that conflicting operations follow a consistent order.

A timestamp-based protocol is a concurrency control technique that decides the order of transaction execution using timestamps instead of locks. Each transaction gets a unique timestamp when it starts, and the system ensures the final outcome is equivalent to a serial schedule in timestamp order. Older transactions (smaller timestamps) are given priority.

Timestamps can come from the system clock or a logical counter.

Each transaction T is assigned a unique timestamp TS(T) when it enters the system. This timestamp determines the transaction's place in the execution order.

Timestamp Ordering Protocol

The Timestamp Ordering Protocol enforces that older transactions (with smaller timestamps) are given higher priority. This prevents conflicts and ensures the execution is serializable and deadlock-free.

For example:

- If Transaction T1 enters the system first, it gets a timestamp TS(T1) = 007 (assumption).
- If Transaction T2 enters after T1, it gets a timestamp TS(T2) = 009 (assumption).

This means T1 is "older" than T2 and T1 should execute before T2 to maintain consistency.

- TS(Ti): timestamp of transaction Ti, assigned when it enters the system.
- R-timestamp(Q): the largest timestamp of any transaction that successfully read data item Q.
- W-timestamp(Q): the largest timestamp of any transaction that successfully wrote Q.

Basic Timestamp Ordering (TO) Protocol

When Ti issues READ(Q):

- If TS(Ti) < W-timestamp(Q): Ti would read a value already overwritten by a younger transaction. Reject and rollback Ti.
- Otherwise: allow the read, and set R-timestamp(Q) = max(R-timestamp(Q), TS(Ti)).

When Ti issues WRITE(Q):

- If TS(Ti) < R-timestamp(Q): a younger transaction has already read the value Ti is about to write. Reject and rollback Ti.
- If TS(Ti) < W-timestamp(Q): Ti is trying to write an obsolete value. Reject and rollback Ti.
- Otherwise: allow the write, and set W-timestamp(Q) = TS(Ti).

A rolled-back transaction restarts with a new timestamp.

Thomas' Write Rule

A modification of the basic protocol. When TS(Ti) < W-timestamp(Q) on a write, instead of rolling back, the obsolete write is simply ignored. This is safe because a later transaction has already overwritten that value, and no one needs Ti's version. It allows some view-serializable schedules that basic TO would reject, reducing needless rollbacks.

Example

Let TS(T1) = 10 and TS(T2) = 20.

- T2 writes Q, so W-timestamp(Q) = 20.
- T1 then tries to read Q. Since 10 < 20, T1 would read a value "from the future," so T1 is rolled back and restarted with a new timestamp.

Importance in DBMS

1. Guarantees serializability: all accepted schedules are equivalent to a serial order based on timestamps.
2. Deadlock-free: transactions never wait for locks, so no wait-for cycles can form. This is a major advantage over locking protocols.
3. Conflict ordering is automatic: conflicting operations are always executed in timestamp order, so no separate precedence-graph checks are needed.

4. No lock management overhead: no lock tables or lock acquisition and release.
5. Good for read-heavy workloads: with few conflicts, rollbacks are rare and concurrency is high.
6. Foundation for advanced techniques: it underlies Multiversion Timestamp Ordering (MVCC), used in systems like PostgreSQL and Oracle, where readers never block writers.

Timestamp-based protocols are one family of concurrency control techniques. They order transactions by start time and reject (roll back) any operation that violates that order. The related concepts below differ in how they enforce serializability.

The Concepts

1. Timestamp Ordering (TO) Protocol

Serializability order is fixed in advance by timestamps. Conflicting operations are checked against R-timestamp and W-timestamp of each data item, and violators are rolled back. There are no locks and no waiting.

2. Lock-Based Protocols (including Two-Phase Locking, 2PL)

Transactions acquire shared or exclusive locks before accessing data. The serial order is determined dynamically by the order in which locks are acquired (for 2PL, the lock point). Transactions wait when a lock is unavailable, so deadlock is possible and must be handled.

3. Validation-Based (Optimistic) Protocols

Transactions run freely in three phases: read, validation, write. Conflict checking is postponed until the end, when the transaction is validated against others. It uses timestamps too (start, validation, finish), but the check happens once at commit, not on every read/write. Best when conflicts are very rare.

4. Multiversion Concurrency Control (MVCC)

Keeps multiple versions of each data item. Multiversion Timestamp Ordering is an extension of timestamp ordering: reads always succeed by selecting the right version, so readers never block or abort. Basic TO keeps a single version, so late reads cause rollbacks.

5. Thomas' Write Rule

Not a separate protocol but a refinement of TO. Obsolete writes are ignored rather than causing rollback, allowing some view-serializable schedules that basic TO rejects.

6. Graph-Based / Tree Protocols

Require prior knowledge of data access order (a partial ordering of items). Locks are acquired following a tree structure. Deadlock-free, but limited to cases where such an ordering is known.

Validation Based Protocol in DBMS

Validation-Based Protocol, also known as Optimistic Concurrency Control, is a technique in DBMS used to manage concurrent transactions without locking data during execution. It assumes that conflicts between transactions are rare and delays conflict detection until the validation phase just before committing.

Why It's Called Optimistic Concurrency Control

- Assumes low interference among transactions.
- Does not check for conflicts during execution.
- All updates are done on local copies.
- At the end, a validation check ensures data consistency before writing to the database.

Phases of Validation-Based Protocol

This protocol operates in three main phases:

1. Read Phase:

- The transaction reads data and performs calculations using temporary (local) variables.
- No changes are made to the database yet.

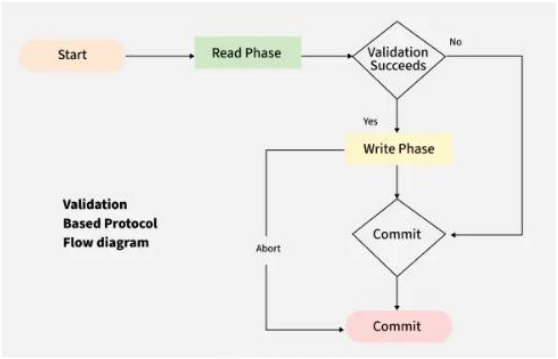
2. Validation Phase:

- Before committing, the transaction checks for conflicts with other concurrently running transactions.
- If no conflict is found (i.e., serializability is preserved), it proceeds to the write phase.
- If validation fails, the transaction is aborted and restarted.

3. Write Phase:

- If validation passes, the transaction writes changes to the database.
- If validation fails, the transaction is aborted and may be restarted.

The diagram below shows the working of the transaction based protocol.



Validation Based protocol

Validation-based protocol delays conflict checks until the end. It works well in systems where most transactions don't interfere with each other.

- In low-conflict environments → high success rate.
- In high-conflict environments → frequent rollbacks, reduced efficiency.

Timestamps Used

Validation based protocols often use timestamps to manage the serialization order of transactions. Each transaction Ti is associated with three important timestamps:

- Start (Ti): The time when transaction Ti begins its execution (read phase).
- Validation (Ti): Time when Ti starts validation. Often used as its logical timestamp for serialization.
- Finish (Ti): The time when transaction Ti completes its write phase (if validation is successful).

Two more terms that we need to know are:

- Write_set: A set of transaction that contains all the write operations that T_i performs.
- Read_set: A set of transaction that contains all the read operations that T_i performs.

Example:

Let's consider two transactions where Tj starts before Ti, since $TS(T_j) < TS(T_i)$ so the validation phase succeeds in the Schedule A. It's noted that the final write operations to the database are performed only after the validation of both Ti and T_j. Since T_i reads the old values of x(12) and y(15) while print (x+y) operation unless final write operation take place.

Time	Tj (Earlier TS)	Ti (Later TS)	Notes
T0		r(x) → x = 12	Ti starts and reads x = 12
T1	r(x) → x = 12		Tj starts and reads x = 12
T2	x = x - 10		Tj computes new value for x
T3		r(y) → y = 15	Ti reads y
T4		y = y + 10	Ti computes new value for y
T5	r(x)		Tj re-reads x (local copy still 12)

Time	Tj (Earlier TS)	Ti (Later TS)	Notes
T6		<validate>	Ti enters validation phase
T7		print(x + y) → 27	Uses local x = 12, y = 15
T8	<validate>		Tj enters validation phase
T9	w(x)		Tj writes x
T10	w(y)		Tj writes y

Analysis

- Both Tj and Ti read the old values (x = 12, y = 15) before any writes occur.
- Ti validates before Tj writes, so it reads consistent, unmodified data.
- Tj's writes occur only after validation, so there is no conflict during Ti's validation phase.

For Ti to successfully validate against Tj, the following two conditions should meet:

1. Condition 1 applies: Finish(Tj) < Start(Ti) is not true.
2. But Condition 2 applies: Ti's read set does not intersect with Tj's write set at the time of validation.

Multiversion Schemes in DBMS

A multiversion scheme is a concurrency control technique that keeps multiple versions of each data item. Instead of overwriting a value on update, the system creates a new version and retains the old ones. When a transaction reads, the system picks the appropriate version for it, so reads are never rejected or made to wait.

Each version Qk of data item Q stores:

- Content: the value of that version
- W-timestamp(Qk): timestamp of the transaction that created it
- R-timestamp(Qk): the largest timestamp of any transaction that successfully read it

Main Types

1. Multiversion Timestamp Ordering: uses timestamps to choose versions.
2. Multiversion Two-Phase Locking: read-only transactions use timestamps (snapshots); update transactions use 2PL.

3. Snapshot Isolation: each transaction sees a consistent snapshot as of its start time (used in PostgreSQL, Oracle).

Multiversion Timestamp Ordering Rules

Let Qk be the version of Q with the largest W-timestamp ≤ TS(Ti).

- Read(Q): return the value of Qk. Always succeeds. Update R-timestamp(Qk) = max(R-timestamp(Qk), TS(Ti)).
- Write(Q):
    - If TS(Ti) < R-timestamp(Qk): a younger transaction already read Qk, so roll back Ti.
    - Otherwise, create a new version of Q with W-timestamp = R-timestamp = TS(Ti).

Example

Let there be two transactions: TS(T1) = 10 and TS(T2) = 20. Data item Q starts with version Q0 = 100 (W-ts = 0, R-ts = 0).

1. T2 writes Q = 150. Qk is Q0, and TS(T2)=20 ≥ R-ts(Q0)=0, so a new version Q20 = 150 is created.
2. T1 reads Q. Under basic timestamp ordering, T1 (TS=10) would be rolled back because a younger transaction wrote Q. In the multiversion scheme, the system selects the version with the largest W-ts ≤ 10, which is Q0 = 100. The read succeeds, and R-ts(Q0) becomes 10.
3. T1 now tries to write Q = 120. Qk is Q0 (largest W-ts ≤ 10). Check: TS(T1)=10 < R-ts(Q0)=10? No, so the write is allowed, creating version Q10 = 120.

Version	Value	W-ts	R-ts
Q0	100	0	10
Q10	120	10	10
Q20	150	20	20

The read that would have caused a rollback in single-version timestamp ordering simply succeeds here.

Importance

- Readers never block or abort, and writers don't block readers.
- Higher concurrency, especially for read-heavy workloads and long read-only queries.
- Deadlock-free (in the timestamp version).
- Basis of modern systems such as PostgreSQL, Oracle, and MySQL InnoDB.

Drawbacks

- Storage overhead from keeping many versions.
- Garbage collection is needed to delete versions no active transaction can need.

- Writes can still be rolled back (when a younger transaction has read the version).
- Recoverability needs extra care: a transaction should commit only after the transactions whose versions it read have committed.

Multiversion Schemes: A Practical Example (Bank Account System)

Scenario

A bank database has an accounts table. While a nightly audit report (a long read-only query) is running, customers keep transferring money. We'll see how multiversion concurrency control (snapshot isolation style, as in PostgreSQL/Oracle) handles this.

Initial data:

acc_id	owner	balance
A	Anita	1000
B	Basil	2000

Total money in the bank = 3000.

Transactions

- T1 (Audit report): read-only, starts at time 10. Computes SUM(balance).
- T2 (Transfer): starts at time 20. Moves 500 from A to B.

Step-by-Step Execution

Time	Transaction	Action	Versions created / selected
10	T1	BEGIN (snapshot taken)	Sees only data committed up to time 10
11	T1	SELECT balance FROM accounts WHERE acc_id='A'	Reads A_0 = 1000
20	T2	BEGIN	
21	T2	UPDATE accounts SET balance = 500 WHERE acc_id='A'	Creates new version A_{20} = 500 (A_0 kept)
22	T2	UPDATE accounts SET balance = 2500 WHERE acc_id='B'	Creates new version B_{20} = 2500 (B_0 kept)
23	T2	COMMIT	A_{20} and B_{20} become visible to *new* transactions
24	T1	SELECT balance FROM accounts WHERE acc_id='B'	Snapshot is at time 10, so reads B_0 = 2000

Time	Transaction	Action	Versions created / selected
25	T1	Computes sum: 1000 + 2000 = 3000, COMMIT	

Version Store After the Transactions

Item	Value	Created by	Visible to
A_o	1000	Initial	Transactions started before T2's commit (e.g., T1)
A_{2o}	500	T2	Transactions started after T2's commit
B_o	2000	Initial	Same as above
B_{2o}	2500	T2	Same as above

What Would Go Wrong Without Multiversioning?

With a single-version, lock-based system:

- Option 1: T1 holds read locks. T2's update is blocked until the long audit finishes, so customers can't transfer money.
- Option 2: T1 doesn't hold locks. T1 reads A = 1000 (old), then T2 commits, then T1 reads B = 2500 (new). The sum is 3500, which is wrong (an inconsistent read).

With multiversion, neither problem occurs: T1 sees a consistent 3000, and T2 proceeds without waiting.

Where Writes Can Still Conflict

Suppose T3 and T2 both try to update account A concurrently:

- T2 updates A and holds the new version (uncommitted).
- T3 tries to update A too. Under snapshot isolation (first-updater-wins), T3 waits or aborts because another transaction already modified A.

So readers never block, but write-write conflicts are still resolved by waiting or rollback.

Same Idea in Real SQL (PostgreSQL)

```
-- Session 1 (Audit)

BEGIN ISOLATION LEVEL REPEATABLE READ;

SELECT SUM(balance) FROM accounts;      -- returns 3000


-- Session 2 (Transfer), runs meanwhile

BEGIN;

UPDATE accounts SET balance = balance - 500 WHERE acc_id = 'A';

UPDATE accounts SET balance = balance + 500 WHERE acc_id = 'B';
```

COMMIT;

-- Session 1 again

SELECT SUM(balance) FROM accounts; -- still returns 3000 (same snapshot)

COMMIT;

-- New session after both commits

SELECT SUM(balance) FROM accounts; -- returns 3000 (new versions: 500 + 2500)

PostgreSQL implements this by storing each row version with hidden fields xmin (creating transaction) and xmax (deleting/updating transaction). A background process, VACUUM, removes old versions no longer needed.